



Degree Project in Mathematics

Second cycle, 30 credits

Optimal Order Placement Using Markov Models of Limit Order Books

MAX OLIVEBERG

Optimal Order Placement Using Markov Models of Limit Order Books

MAX OLIVEBERG

Master's Programme, Mathematics, 120 credits

Date: June 2, 2023

Supervisors: Jonas Kiessling, Anders Szepessy

Examiner: Anders Szepessy

School of Engineering Sciences

Host company: Qubos Systematic AB

Swedish title: Optimal Orderläggning med Markovmodeller av Orderböcker

Abstract

We study optimal order placement in a limit order book. By modelling the limit order book dynamics as a Markov chain, we can frame the purchase of a single share as a Markov Decision Process. Within the framework of the model, we can estimate optimal decision policies numerically. The trade rate is varied using a running cost control variable. The optimal policy is found to result in a lower cost of trading as a function of the trade rate compared to a market order only strategy.

Keywords

Optimal order placement, Limit order book, Markov

Sammanfattning

Vi studerar optimal orderläggning i en *limit* orderbok. Genom att modellera dynamiken av inkommande ordrar som en Markov kedja så kan vi formulera optimal orderläggning som en *Markov Decision Process*. Inom ramverket av modellen så kan vi skatta optimala strategier numeriskt. En löpande kostnad används som en kontrollvariabel för handelstakten av den optimala strategin. Vi finner att den optimala strategin resulterar i en lägre handelskostnad som funktion av deltagande jämfört med en marknadsorder strategi.

Nyckelord

Optimal orderläggning, Orderbok, Markov

Acknowledgments

First and foremost, I would like to express my sincere gratitude to Jonas Kiessling and Anders Szepessy for their supervision of this thesis. Jonas patiently answered any of my questions, be it regarding quantitative finance or anything else. Anders provided invaluable advice on the project and academic work in general. Thank you both!

I would also like to acknowledge the support and encouragement from Jakob Stigenberg, who always made time to discuss implementation, methodology, and VR-headsets.

Finally, thank you Ebba for your support while writing this thesis.

Stockholm, June 2023

Max Oliveberg

Contents

1	Introduction	1
1.1	The Limit Order Book	2
1.1.1	Modelling the Limit Order Book	2
1.2	Markov Processes & Decision Policies	4
1.3	Value Iteration	8
2	A Model for the Limit Order Book	11
2.1	Markov Model	11
2.1.1	Model considerations	15
2.1.2	Introducing agent orders to the state	16
2.2	Evaluation Framework	19
2.2.1	Sampling a Poisson process	20
2.3	Reference price	21
2.4	Historical Data	24
2.4.1	Reduced state representation	24
3	Optimal Order Placement on Synthetic Data Using the Matrix State Representation	27
3.1	Experimental setup	27
3.1.1	Decision process on the matrix state representation	27
3.1.1.1	Transitions resulting from agent orders	29
3.2	Results	29
3.2.1	Observed behaviour of the optimal strategy	29
4	Optimal Order Placement on Synthetic and Historic Data Using the Reduced State Representation	33
4.1	Experimental setup	33
4.1.1	Decision process on the reduced state representation	33
4.1.2	Evaluation metrics	35
4.1.3	State space, data sequences and initial states	35

4.2	Results	36
4.2.1	Results on synthetic data	36
4.2.2	Results on historic data	37
4.2.3	Discussion	41
5	Conclusions and Future work	43
	References	45
A	Markov Processes	47
A.1	Potentials	47
A.2	State Indexation	48
B	Historical Data	50
B.1	Order Matching	50
B.1.1	A simple order matching algorithm	51
B.2	'Synthetic Layer' on Historical Data	54
B.3	Parameter fitting	55
B.4	Historic data used	56
C	Simple Strategies	59
C.1	Market Order	59
C.2	Bid Plus One	59

List of Figures

1.1	A representation of a LOB, along with examples.	3
3.1	Results for synthetic data, using the matrix state representation.	32
4.1	Results on synthetic data.	38
4.2	Results on historic data.	39
4.3	Comparison between synthetic and historic results.	40
B.1	A sample of the distributions, from the date <i>09-01-2023</i>	58

List of Tables

2.1	A table over all non-zero intensities.	14
B.1	A table over the historic data selected.	57

Notation and terminology

We denote the state space of a Markov chain as S , and the states within that space $s \in S$. Different states may be denoted by either index: s_i, s_j , or apostrophe: s, s' . Details regarding implementing an indexation are discussed in Appendix A.2.

Semantic sub and superscript are used to denote various parameters. We use $\{\mathbf{B}, \mathbf{S}\}$ to denote *buy* and *sell* respectively. Order types are denoted as $\{\mathbf{L}, \mathbf{M}, \mathbf{C}\}$ for *limit*, *market* and *cancel*.

Actions defined for a state s , $a \in A(s)$, may be denoted with a semantic subscript, such as a_L for the action of placing a limit order. If no parameters are further specified, then the action is any limit order action $a_L \in A(s)$.

We let e_i denote the base vector in $\mathbf{R}^n$.

We denote random variables as being functions of some set of outcomes Ω , e.g

$$v_T(s) : S \times \Omega \rightarrow \mathbf{R}, \quad (1)$$

and assume the existence of a corresponding σ -algebra Σ , measure $\mathbf{P}$, and filtration $\mathcal{F}_t$.

The abstract entity which takes action in one of our MDPs is referred to as the *agent*. It takes these actions according to some *policy*. All actions are either representations of placing some type of order, or 'waiting' for the state to transition. In the latter case, the transition models another market participant submitting an order to the LOB. Thus, we sometimes refer to such transitions as a *non-agent order* arriving.

Chapter 1

Introduction

In the work, we study the optimal order placement problem in a limit order book. We define this, as proposed by Guo [1], to be how we can optimally place small orders into the limit order book to minimize trading costs. In particular, we study the placement of unit size orders. We extend and implement a Markov model for the limit order book micro-structure dynamics proposed by Hult & Kiessling [2]. The main alterations to [2] is the introduction of a benchmark *reference price*, and the discarding of absolute price in the Markov state. The reduction in state space size resulting from the removal of the absolute price yields a more tractable state space. Longer sequences of states can then be extracted from historic data. The changes together with the numerical experiments, are the main contributions of the study.

The optimal order placement problem, and the related optimal order execution problem*, have been studied extensively in the literature. In [3], reinforcement learning is used on historic order book data in order to execute some inventory of N orders before time T . Similarly, [4] explores several reinforcement learning approaches to optimal order execution in the context of a Markovian order book. Both studies consider the execution of larger orders, while we study the placement and execution of singular orders. In [5], recurring neural networks are proposed as a method for generating synthetic LOB data. This approach is intended to relax the Markovian assumption on the limit order book.

*In [1], the two are distinguished by timescale and focus on the micro-structure of the LOB. It is challenging to make a strict distinction between the two, e.g. in [3] the optimal execution problem is solved with a micro-structure centered approach.

1.1 The Limit Order Book

The most common format for securities trading on exchanges is the *limit order book* (LOB). It consists of a list of all outstanding buy and sell orders for some financial instrument. A market participant can place a *limit order* of some size and price to be appended to the list of outstanding orders. The price of the best offer to buy is referred to as the *best bid*, and the price of the best offer to sell is referred to as the *best ask*. Market participants can at any time cancel any outstanding limit order.

If the order is placed so that its price is higher (lower) or equal to the best ask (bid), then the order is *matched* and a trade occurs. Both the incoming order and counter party order are then removed from the LOB. Placing such an order is referred to as placing a *market order*.

In the stylised LOB studied in this work, prices p are restricted to fixed increments, or *ticks*, of some size δp appropriate to the instrument in question. A participant can only buy (sell) at prices

$$p = N\delta p, \quad (1.1)$$

for integers $N > 0$. Consequently, the order book is divided into levels, with some number of outstanding limit orders at each level. We refer to the number of orders at a price level as the *depth* at that price level. In the work, we will most often refer to prices by some number of ticks i relative to a price p_0 :

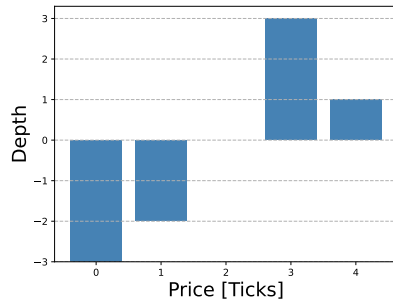
$$p = p_0 + i\delta p. \quad (1.2)$$

When a market order arrives, priority is given by *price-time* priority. The limit orders offering the best deal are matched first. Limit orders on the same price level are prioritised by *first in first out*, and the oldest limit order is given priority over newer orders.

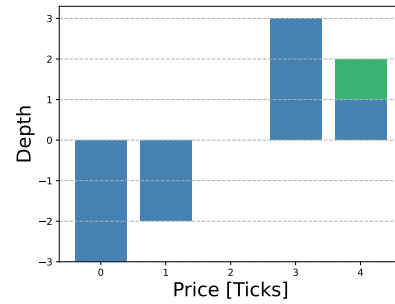
A representation of a LOB is shown in Figure 1.1.

1.1.1 Modelling the Limit Order Book

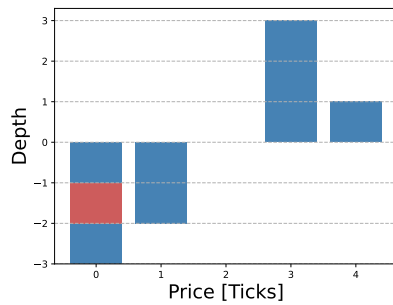
Before we proceed to the specific method and model used in the work, we consider how the LOB can be modelled mathematically. The discussion focuses on a micro-structure model, as opposed to a mean field approach where



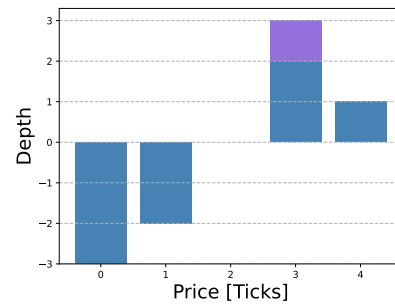
(a) A representation of an order book, with price levels along the x-axis and the corresponding depths along the y-axis. Buy orders are represented as negative.



(b) A limit order arrives at price level $i = 4$, represented by a green block. This increases the depth from one to two.



(c) The order represented by a red block is cancelled, resulting in a depth of two at price level $i = 0$.



(d) A market order arrives at price level $i = 3$, and matches with the limit order represented by a purple block. This lowers the depth at the price level from three to two.

Figure 1.1: A representation of a LOB, along with examples.

only the current price is modelled, e.g the random walk model of [6].

The LOB consists of some finite set of limit orders. If we want to create a micro-structure model for the LOB, it is natural that we represent this set of orders by some state s in a countable state space S , which represents all possible sets of orders. The precise information contained in one of these orders is model dependent. For example, on a real exchange each order originates from a specific market participant. However, we may not be concerned with whom we trade with, only that we have executed a trade. If one stores a limit order of size n as n separate, unit sized limit orders, then the bare minimum information regarding an order is the price, the priority and the direction.

Orders are added to the state through limit orders, and removed via market orders or cancellations. We approximate that all market participants place their orders independently, and we model that there is *almost surely* some time $\delta t > 0$ between two subsequent orders from different market participants. This allows us to view the LOB as random process changing states at discrete times. In the sequel, we will model the various order types as being independent Poisson arrival processes. An alternative approach is to explicitly model the behaviour of every actor on the exchange. An open source framework for this is provided in [7]. Such a model is very fine, but is cumbersome to calibrate.

Our agent can interact with our model by placing a limit order, cancelling a limit order, placing a market order or waiting for another market participant to act. The agents actions may be subject to latency, to model real life delays in communication between an actor and an exchange. In the work, the agent will have zero latency when interacting with the LOB. In Section 5, we give a brief example of how latency can be modelled in the context of the model.

1.2 Markov Processes & Decision Policies

In Section 2.1, we will model the LOB dynamics as a Markov chain. Here, we describe some of the fundamental definitions and properties of Markov processes. More aspects relevant to the work are given in Appendix A. For a more thorough account we refer to [8]. First, we define a Markov process:

Definition 1.2.1 *Let $s(t) : [0, \infty] \times \Omega \rightarrow S$ be a random process that has assumed some values $s(t_0), \dots, s(t_n) \in S$ at times $t_i < t$. We say that $s(t)$ is a*

Markov process if:

$$\mathbf{P}(s(t) = s | s(t_0), \dots, s(t_n)) = \mathbf{P}(S(t) = s | s(t_n)). \quad (1.3)$$

We say that a random process fulfilling Equation (1.3) has the *Markov Property*. An example is a Discrete Time Markov Chain (DTMC). A DTMC is a random process $s(i)$, $i \in \mathbf{N}$, which assumes values in some countable space S . Its transitions is governed by a stochastic matrix:

Definition 1.2.2 A stochastic matrix $P_{s,s'} : S^2 \rightarrow \mathbf{R}$ fulfills

1. $P_{s,s'} \geq 0$,
2. $\sum_{s' \in S} P_{s,s'} = 1$.

Since each row sums to unity, the element $P_{s,s'}$ can be interpreted as the transition probability from state s to state s' . A DTMC can then be defined as:

Definition 1.2.3 Let S be a state space, $P_{s,s'}$ a stochastic matrix on that state space and $s_0 \in S$. A discrete time Markov chain $s(i) : \mathbf{N} \times \Omega \rightarrow S$ is a random sequence such that

1. $s(0) = s_0$,
2. $\mathbf{P}(s(i+1) = s' | s(i) = s) = P_{s,s'}$.

In Section 2, we will model the limit order book as a Continuous Time Markov Chain (CTMC). A CTMC is a random process $s(t)$ with a countable state space S with transition intensities defined by the *rate matrix* $Q_{s,s'}$.

Definition 1.2.4 Let S be a countable state space. A rate matrix, $Q_{s,s'} : S^2 \rightarrow \mathbf{R}$, is a matrix such that:

1. $Q_{s,s'} \geq 0$ for all $s \neq s'$,
2. $|Q_{s,s}| < \infty$,
3. $\sum_{s'} Q_{s,s'} = 0$.

The first and third points in the definition imply that the diagonal of the rate matrix fulfills

$$Q_{s,s} = - \sum_{s' \neq s} Q_{s,s'}. \quad (1.4)$$

The time that the process spends in a state is referred to as the *holding time* $T(s)$. The holding time is a random variable, and is exponentially distributed:

$$T(s) \in \text{Exp}(-Q_{s,s}). \quad (1.5)$$

We can define a continuous time Markov chain as: *

Definition 1.2.5 Let $s(t)$ be a Markov process as per Definition 1.2.1 which takes values in a countable space S , and let $Q_{s,s'}$ be a rate matrix as per Definition 1.2.4. We say that the process $s(t)$ is a Continuous Time Markov Chain if for every time t and some infinitesimal δt , we have

$$\mathbf{P}(s(t + \delta t) = s' | s(t) = s) = I\{s = s'\} + \delta t Q_{s,s'} + o(\delta t). \quad (1.6)$$

Given that we are in some state $s(t)$ at time t , we can calculate the probability of the next state being s' as

$$P_{s,s'} = \frac{Q_{s,s'}}{\sum_{s'' \neq s} Q_{s,s''}} = -\frac{Q_{s,s'}}{Q_{s,s}}, \quad s \neq s'. \quad (1.7)$$

This is the *jump matrix* for the process, and we see that it is a stochastic matrix as per Definition 1.2.2. In the work, we will primarily be concerned with the *jump chain*. The jump chain is the discrete time random process of the states of some CTMC:

Definition 1.2.6 Let $s(t)$ be a continuous time Markov chain with rate matrix $Q_{s,s'}$, and initial value $s(0)$. The jump chain $\bar{s}(n)$ of $s(t)$ is the discrete Markov chain with initial value $\bar{s}(0) = s(0)$ and transition matrix $P_{s,s'}$ as in (1.7).

The reason for the focus on the jump chain is twofold: First, the optimal policy for a Markov Decision Process (MDP) is stationary, i.e independent of time. Second, the optimal strategy for the model in Section 2.1 is not dependent on the transition times, only the jump matrix. The jump matrix is unchanged if we scale all intensities $Q_{s,s'}$ by some scalar c , i.e the transform

$$Q_{s,s'} \rightarrow \frac{Q_{s,s'}}{c} \quad (1.8)$$

leaves (1.7) unchanged but the transition time of (1.5) scaled by a factor $\frac{1}{c}$.[†]

*This is the *infinitesimal* definition. One can equivalently define the CTMC through the *transition probability* and *holding time* definitions. See **Theorem 2.8.2** in [8].

[†]See the remarks at the end of Section 2.1 for an example of how such scaling can be exploited in the context of the model presented.

Despite the focus on the jump chain, we will discuss the transition times where appropriate, such as in Section 2.2.1. Through abuse of notation, we will refer to the jump chain $\bar{s}(n)$ as $s(t)$, with the understanding that the optimal strategy for $\bar{s}(n)$ is, in practical terms, the optimal strategy for $s(t)$ if the agent has zero latency when taking action.

A Markov chain may have some set of terminal states $\partial S \subseteq S$. These are absorbing states, and if the process $s(t)$ reaches the absorbing states it will end:

Definition 1.2.7 *A state s is terminal, $s \in \partial S$, if*

$$\mathbf{P}(s(t + \Delta t) = s | s(t) = s) = 1 \quad (1.9)$$

for all Δt .

To construct a Markov decision process using the state space S as a starting point, we for each state $s_i \in S$ define some finite set of actions $A(s_i)$. Given a state $s_i \in S$, the agent chooses some $a \in A(s_i)$, which in turn transitions the state according to some action-dependent jump matrix. We can write the jump matrix as a function of a , $P_{s,s'}(a)$.

Each action a in state s has an outcome-dependent random reward $R_{s,s'}(a)^*$ which the agent receives. Further, if the process hits some terminal state $s \in \partial S$, then the agent receives the termination reward $v_T(s)$. With the pieces in place, we can define a MDPs:

Definition 1.2.8 *Let S be state space, $s(0) = s_0 \in S$ an initial state, $A(s)$ a non-empty set of actions defined for each $s \in S$ and $P_{s,s'}(a) \in \mathbf{R}$ a transition matrix between states in S , given an action $a \in A(s)$. Let the reward matrix $R_{s,s'}(a)$ be a random variable*

$$R_{s,s'}(a) : \Omega \rightarrow \mathbf{R}, \quad (1.10)$$

and let $v_T(s) : \partial S \times \Omega \rightarrow \mathbf{R}$. We define a Markov decision process as the tuple (S, s_0, A, P, R, v_T) , and the reward for a sequence of actions a_i on a sequence states $s(i)$ as

$$R = \sum_i R_{s_i, s_{i+1}}(a_i) + v_T(s_T). \quad (1.11)$$

*Equivalently, one can instead use a cost $C_{s,s'}(a)$ and instead formulate the optimal policy problem as one of cost minimisation. One can also define the reward as being given when the agent visits some state s , i.e $R(s)$, though this view is less flexible in the context of the work.

In particular, we note that if we pick a sequence of actions, a_i , for sequential states, $s(i)$, then the resulting process is a DTMC. If we have a predetermined action for every state s and time t , we have a policy $\alpha(s, t) \in A(s)$. Using the Markov property, the expected value $V(s, \alpha)$ of the policy $\alpha(s, t)$ in state s is

$$V(s, \alpha, t) = I\{s \notin \partial S\} \mathbf{E}[R_{s,s'}(\alpha(s)) + V(s', \alpha, t+1)] + I\{s \in \partial S\} \mathbf{E}[v_T(s)]. \quad (1.12)$$

This naturally leads to the definition of an *optimal* policy as:

Definition 1.2.9 A policy α^* is optimal if

$$V(s, \alpha^*) \geq V(s, \alpha) \quad (1.13)$$

for all $s \in S$ and policies α .

1.3 Value Iteration

By construction, we will only study strategies which almost surely terminate in finite time. If in addition the rewards are bounded, the expected value in Equation (1.12) is also bounded. Approximating an optimal policy α^* can be achieved with value iteration. In this section, we will give the results relevant to value iteration. Proofs are omitted for brevity, but we give motivation and references where it is relevant.

Consider a MDP, where every policy almost surely terminates in finite time. We have some set of terminal states $\partial S \subseteq S$, and a reward matrix R which is bounded, $|R_{i,j}| \leq R_0$. For each state $s \in S$, define

$$V_0(s) = \begin{cases} v_T(s), & s \in \partial S \\ \inf_{s'} v_T(s'), & \text{otherwise} \end{cases} \quad (1.14)$$

Then, let $V_{n+1}(s)$ be defined by

$$V_{n+1}(s) = \max_{a \in A(s)} \mathbf{E}[R_{s,s'}(a) + V_n(s')]. \quad (1.15)$$

Finally, define V_∞ to be the limit

$$V_\infty(s) = \lim_{n \rightarrow \infty} V_n(s). \quad (1.16)$$

The sequence $V_n(s)$ is monotone increasing for all $s \in S$:

Proposition 1.3.1 *Let $V_n(s)$ be as in (1.15). Then*

$$V_{n+1}(s) \geq V_n(s), \quad (1.17)$$

for all n, s .

The monotone increasing behaviour can be shown through induction.

Proposition 1.3.2 *For all states $s \in S$,*

$$V_\infty(s) = \lim_{n \rightarrow \infty} V_n \quad (1.18)$$

is bounded.

That the limit $V_\infty(s)$ is bounded follows directly from bounded rewards, and that all strategies almost surely terminate in finite time.

Proposition 1.3.3 *The policy $\alpha_\infty(s)$, which maximises the expected value in $V_\infty(s)$, is an optimal, stationary policy.*

This is a slight reformulation of **Lemma 5.4.2** and **Theorem 5.4.3** in [8], or **Theorem 4.3** in [2]. Since the optimal policy, α^* , is stationary, the problem is reduced to finding a single, optimal action for every state, $s \in S$.

With these propositions, we can estimate the optimal policy by calculating $V_{n+1}(s)$ until

$$\max_{s \in S} (V_{n+1}(s) - V_n(s)) < \epsilon, \quad (1.19)$$

for a suitable threshold $\epsilon > 0$. Our estimation of the optimal policy, α^* , is then the argument in each state s . This process is referred to as *value iteration*, and is the method used to estimate optimal policies in the work.

Now that we have familiarised ourselves with MDPs, we are ready to present the model for the LOB.

Chapter 2

A Model for the Limit Order Book

In this chapter, we model the Limit Order Book as a Continuous Time Markov Chain. In the first section, we present a model suggested by [2] and then modify the state representation to make it more flexible with regards to agent orders. In the sequel, we consider how to simulate and value policies. Finally, we adapt the model to constraints imposed by the available data.

2.1 Markov Model

The work here is based off the LOB model suggested by Hult & Kiessling in [2]. The order book state is represented by a vector:

$$s = \begin{bmatrix} s_0 \\ \vdots \\ s_{n-1} \end{bmatrix}. \quad (2.1)$$

Each dimension represents the number of orders, or *depth*, at ascending price levels relative to some price p_0 , with the lowest price level being at position $i = 0$ and the highest at $i = n - 1$. The entry in each dimension represents the order depth at the corresponding price level, with the convention that buy orders are represented as negative numbers. As an example, for a three-level order book model, the state $s = [-1, 0, 1]^T$ signifies that there is a limit buy order at price level $i = 0$, a limit sell order at $i = 2$ and that the price level in-between is empty. The state $s = [1, -1, 1]^T$ represent a crossed order book, as the best bid is higher than the best ask. A crossed book is not a

valid state within our model. We denote the set of all valid states by S . The subset $\partial S \subset S$ is called the set of *terminal states*. A terminal state is a state corresponding to the conclusion of a trading strategy, for instance the execution of an order in a single order execution strategy.

In addition to the variable orders in the state, we assume that one can always sell a share at price $i = -1$, and always buy a share at price n . These assumptions constrain the best bid i_B and best ask i_A :

$$-1 \leq i_B < i_A \leq n. \quad (2.2)$$

The best bid is the highest price a market participant is willing to purchase a share for, i.e the outstanding limit buy order with the highest price. Likewise, the best ask is the lowest price a market participant is willing to sell a share for. We can define the bid price as

$$i_B = \max_i (\{i : s_i < 0\} \cup \{i = -1\}), \quad (2.3)$$

and the ask price as

$$i_A = \min_i (\{i : s_i > 0\} \cup \{i = n\}). \quad (2.4)$$

Next, we assume that we can observe and react to each individual order arriving to the order book. The orders are modelled as being either a limit order, a market order or a cancellation arriving at a specific price level i . This limits transitions to the general form

$$x \rightarrow x \pm ke_i, \quad (2.5)$$

where k is the order size and e_i is a unit vector. We will model the three order types as independent Poisson arrival processes. These are subject to some constraints: For example, a cancellation can not arrive to an empty price level nor can a limit sell order arrive below the bid.

Limit Orders: Limit buy orders arrive at price levels $i < i_A$. Limit sell orders only arrive at price levels $i > i_B$. A limit order results in a transition

$$s \rightarrow \begin{cases} s - ke_i, & i < i_A \\ s + ke_i, & i > i_B \end{cases}. \quad (2.6)$$

The intensity of limit buy (sell) orders are modelled as proportional to the distance to the ask (bid). Further, there is some distribution p_k governing the proportion of limit orders having size k , the specifics of which we will discuss towards the end of this section. Limit buy orders of size k arrive at price i with intensity

$$\lambda = p_k \lambda_L^B(i_A - i), \quad i < i_A, \quad (2.7)$$

and limit sell orders arrive with intensity

$$\lambda = p_k \lambda_L^S(i - i_B), \quad i > i_B, \quad (2.8)$$

where λ_L^B and λ_L^S are parameters.

Market orders: Market sell orders are modelled as arriving precisely at the bid i_B , and market buy orders arrive precisely at the ask i_A :

$$s \rightarrow \begin{cases} s + ke_{i_B} \\ s - ke_{i_A} \end{cases}. \quad (2.9)$$

The intensity of market orders is modelled as being independent of the state, with sell orders of size k arriving with intensity

$$\lambda = q_k \lambda_M^S, \quad (2.10)$$

and buy orders arrive with intensity

$$\lambda = q_k \lambda_M^B, \quad (2.11)$$

for parameters λ_M^S, λ_M^B . Here, q_k is, similar to p_k , a distribution governing the sizes of market orders. We have that

$$\sum_k q_k \lambda_M^B = \lambda_M^B. \quad (2.12)$$

As a result, we can directly interpret λ_M^B and λ_M^S as the intensity of market orders per unit of time.

Cancellations: Cancellations of limit buy orders arrive at price levels $i \leq i_B$ such that $|s_i| > 0$. Cancellations of limit sell orders analogously only occur at

price levels $i \geq i_A$ where $|s_i| > 0$. A cancel order transition takes the form

$$s \rightarrow \begin{cases} s + e_i, & s_i < 0, i \leq i_B \\ s - e_i, & s_i > 0, i \geq i_A \end{cases}. \quad (2.13)$$

Limit orders *decay* exponentially, with a rate proportional to their distance from the bid (ask). The intensity for limit buy order cancellations is

$$\lambda = \lambda_C^B(i_A - i)|x_i|, \quad (2.14)$$

and for limit sell orders

$$\lambda = \lambda_C^S(i - i_B)|x_i|. \quad (2.15)$$

In [2], cancellations are assumed to only be of unit size, i.e $k = 1$. We will repeat this assumption, to simplify transition intensities when dealing with agent orders in a later section, and to keep the decay-like form of the intensity.

It is worth noting that in the absence of agent orders we have no reason to distinguish market orders of unit size and cancellations at the bid/ask. The net intensity for the transition $x \rightarrow x + e_{i_B}$ is the sum of the intensity of both events

$$\lambda = \lambda_C^B(i_A - i_B)|x_{i_B}| + q_1 \lambda_M^S. \quad (2.16)$$

The intensities are summarised in Table 2.1.

Type	Direction	Price	Size	λ
Limit	Buy	$i < i_A$	k	$p_k \lambda_L^B(i_A - i)$
Limit	Sell	$i > i_B$	k	$p_k \lambda_L^S(i - i_B)$
Market	Buy	i_A	k	$q_k \lambda_M^B$
Market	Sell	i_B	k	$q_k \lambda_M^S$
Cancel	Buy	$i \leq i_B$	1	$\lambda_C^B(i_A - i) x_i $
Cancel	Sell	$i \geq i_A$	1	$\lambda_C^S(i - i_B) x_i $

Table 2.1: A table over all non-zero intensities.

Discrete exponential distributions were used in [2] for p_k and q_k ,

$$p_k = (e^{\alpha_L} - 1)e^{k\alpha_L}, \quad q_k = (e^{\alpha_M} - 1)e^{k\alpha_M} \quad (2.17)$$

where α_L, α_M are parameters. We will use this distribution for our synthetic results in Section 3 and then replace it with a geometric distribution for a closer

fit to historic data in Section 4.

2.1.1 Model considerations

To obtain a finite state space S , one has to limit the maximum depth. In the work, this is achieved by limiting $|s_i| \leq D$. Since the probability of an order size of k decreases rapidly for both the geometric and exponential distributions, truncating the order size, such that $|k| \leq K$ for some K , improves computational performance.

The main drawback of the method is the exponential growth of the state space with the number of price levels n under consideration

$$|S| \sim (2D + 1)^n. \quad (2.18)$$

The possible transitions grows much slower as $\sim \mathcal{O}(Kn)$. As a result, the jump matrix P becomes increasingly sparse as the dimension of the order book increases.

Before we move on to the agent orders, there is an interesting normalisation which can be done when working with the embedded Markov process. Since all intensities depend on some parameter λ_i^j linearly, one can normalise them by a non-zero parameter λ_0 :

$$\Lambda = (\lambda_0, \dots, \lambda_n) \rightarrow (1, \dots, \frac{\lambda_n}{\lambda_0}) = \Lambda/\lambda_0, \quad (2.19)$$

where the indexation is an arbitrary ordering of the λ_i^j parameters such that λ_0 is non-zero. Doing this will affect the average transition time T of the jump process by a factor of λ_0^{-1}

$$\mathbf{E}[T] = \sum \lambda_i \rightarrow \sum \frac{\lambda_i}{\lambda_0}, \quad (2.20)$$

yet leave the jump matrix $P_{s,s'}$ unchanged. A strategy which is optimal for the parameters Λ is still optimal for the parameters Λ/λ_0 . If we also want to include the case where our normalising parameter $\lambda_0 = 0$, we can view the parameter space of Λ as being the projective space P^n . These considerations are useful when trying to reduce the computational complexity, as one parameter is effectively removed.

2.1.2 Introducing agent orders to the state

In [2], for a keep-or-cancel strategy, it was suggested that the price i of an agent order is a hyper parameter and that only the position in the execution queue enters the state explicitly as y , i.e

$$s' = (s, y), \quad (2.21)$$

where $y \geq 0$ is an integer. The agents order itself is included in y , so that $y = 1$ means that the order is next in line to be executed and that $y = 0$ implies that the order has been matched. The MDP then entailed either waiting for the order to execute, cancel it, or place a market order if some conditions were fulfilled. The MDP terminated once the order was executed or cancelled.

To generalise to a case where the agent can have a limit order at any price level, an additional state variable j was added, representing the price level of the limit order:

$$s' = (s, y, j). \quad (2.22)$$

To handle cancellations at the agent price level, we have to sample which order is cancelled according to some scheme. The method used here is to sample which order is cancelled uniformly.

Here, we will reformulate the state representation in a manner which simplifies the book keeping involved in tracking the agents order when implementing the dynamics in code. This is only an exercise in representation, and does not alter the model dynamics.

Let the state be represented by a tuple of a $n \times m$ order book state matrix x and an optional n -vector p representing the agent portfolio

$$s = (p, x). \quad (2.23)$$

The $n \times m$ -matrix x represents the order book state. It is best illustrated with an example, here of a 3×3 -matrix

$$x = \begin{bmatrix} -2 & -2 & -1 \\ 1 & 0 & 0 \\ 0 & 2 & 0 \end{bmatrix}. \quad (2.24)$$

Similarly to the previous vector representation, the number of rows equals the number of price levels in the model. The sum of each row is the depth at the corresponding price level, meaning that we can recover the previous state representation by multiplying x with the n -vector of ones 1^n

$$s_{\text{old}} = x1^n. \quad (2.25)$$

We can then obtain the bid level i_B and ask level i_A in the same manner as Equations (2.3) and (2.4). In the example of Equation (2.24), we would have $i_B = 0, i_A = 1$.

If only the first entry of a row is non-zero, then there are no agent orders and this entry is the depth. In Equation (2.24), we see that at price level $i = 1$, there is only a singular limit sell order, and this order does not belong to the agent.

Following columns represent a slot for a agent order at each price level. If an entry is zero, then there is no agent order. Otherwise, the entry is the number of orders between the order and the next agent order further up the queue, *including* the order itself.

If there are no agent orders at higher column indices, then the entry is the orders position in the execution queue. Having an agent order at a price level changes the meaning of the first column to be the amount of orders behind the lowest priority agent order. In Equation (2.24)

- The agent has two limit buy orders at price level $i_B = 0$. The first is next in line to execute, and there is a single non-agent order between the two. Behind the second order, there are two additional non-agent orders.
- The agent has no orders at the ask $i_A = 1$, where there is a single limit sell order.
- Above the ask, the agent has a single order. There is also a single non-agent order ahead of it, and none behind.

The matrix representation makes the model comparable with the representation of Equation (2.1). For example, we can now write all transitions to x as

$$x \rightarrow x + ke_{i,j}. \quad (2.26)$$

This notation simplifies implementation. The important caveat to this is market orders, since these can impact multiple entries on a row. Through abuse of notation, we denote market order transitions as in Equation (2.26), with the understanding that any 'spill-over' is deducted from lower column indices*.

Next, we note that the various transitions occur at distinct positions in the matrix:

- Limit orders always arrive at the first column, $x \rightarrow x \pm ke_{i,0}$.
- When a market order arrives, we always begin deducting from the right-most non-zero column.
- Cancel orders arrive at an intensity proportionally to the entry in the column, though we must deduct any agent orders from the depth.
- Adding or removing agent orders amounts to shifting the row at the corresponding price index, and adding/subtracting the agent order from the depth at the second/first column.

The intensities per order type are not changed from Table 2.1, though we have to adopt them to the new state representation. In particular, the intensity of a limit buy cancellation arriving at position $e_{i,j}$, $i \leq i_B$, in the matrix is

$$\lambda = \lambda_{\mathbf{C}}^{\mathbf{B}}(i_A - i)(|x_{i,j}| - I\{j > 0\}), \quad (2.27)$$

where $\lambda_{\mathbf{C}}^{\mathbf{B}}$ is the *cancel-buy* parameter of Table 2.1, and

$$I\{j > 0\} = \begin{cases} 1, & j > 0 \\ 0, & j = 0 \end{cases} \quad (2.28)$$

deducts the agent order from the depth for $j > 0$.

Now, we are ready to turn our attention to the agents position. The n -vector p represents the agents net position, with the index of an entry denoting the

*This can be formalised by defining an equivalence relation on S , though we will omit this here for brevity.

trade price and the sign the direction of the trade. For example, the vector

$$p = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \quad (2.29)$$

implies that the agent has purchased one share at price level $i = 0$ and sold one at price level $i = 2$.

2.2 Evaluation Framework

We developed an algorithmic framework to facilitate the testing and evaluation of strategies. The structure is similar to the *Agent-Environment Interface* of [9], though tailored to the stylized LOB of the work.

From the agents point of view, an episode would proceed as follows: First, we are shown some state s_i . Then, we react to this state by taking some action a . As a result of this action, the state transitions to some new state, $s_i \rightarrow s_j$ and we receive reward R_{s_i, s_j} . This process repeats until we hit a state $s \in \partial S$.

Concretely, an action in our case would entail either placing an order, waiting for the state to transition or terminating. We refer to a decision policy as a *strategy*, in reference to trading strategies. A stationary strategy $\alpha(s)$ returns an action, a , as a function of the observed state, s , i.e

$$\alpha(s) = a, \quad (2.30)$$

or in-code as the Python interface* given in Listing 2.1.

Listing 2.1: IStrategy interface

```
class IStrategy:
    def evaluate(self, state: list[float],
               elapsed_time: int) ->
        (StrategyState, Optional[list[Order]]):
        raise NotImplementedError
```

Upon the termination of an episode, we receive a value representing the performance of the strategy. We save an in-depth discussion of the value of a

*The code example has support for non-stationary strategies, through the argument **elapsed_time**. **StrategyState** is an enum to communicate whether a strategy has terminated, and **Order** is a struct containing order information.

strategy for Section 2.3 and for the moment assume that such a value exists, and can be calculated upon termination.

Next, we define a *market* as the counter party to the strategy, responsible for transitioning the state. The market need not transition the state by sampling a Markov process. For instance, we can transition the state using historic data. We represent this as the interface of Listing 2.2*.

Listing 2.2: IMarket interface

```
class IMarket:
    def tick(self, orders: Optional[list[Order]]) ->
        (list[float], int):
        raise NotImplementedError
```

Once one has implemented a strategy and a market, simulations can be run by passing the orders and states back and forth between the two objects.

2.2.1 Sampling a Poisson process

For every state s_i , we have some intensities $\lambda_{i,j}$ for transitions to other states $\{s_j\}$. The transition times are independent Poisson distributions, and one can implement a simple sampling algorithm as follows:

1. For each j such that $\lambda_{i,j} > 0$, sample $T_j \in \text{Exp}(\lambda_{i,j})$.
2. Let the new state be s_j , such that $j = \arg \min_j \{T_j\}$.
3. If the time is of interest, increment the system time by T_j .

This requires us to perform $\sim \mathcal{O}(mn)$ exponential samples per transition, where $n \times m$ are the state-matrix dimensions. One can exploit some properties of the exponential distribution to create a more computationally efficient scheme. If we have a finite set of IID variables $T_j \in \text{Exp}(\lambda_{i,j})$, then

$$\min_j (T_j) \in \text{Exp}\left(\sum_j \lambda_{i,j}\right). \quad (2.31)$$

Next, we have that

$$\mathbf{P}(T_j < T_k, \forall k \neq j) = \frac{\lambda_{i,j}}{\sum_\ell \lambda_{i,\ell}} \propto \lambda_{i,j}. \quad (2.32)$$

*The **tick** function returns an integer, representing the system time.

Thus, we can

1. Sample the outcome s_j with weights $\omega_j \propto \lambda_{i,j}$
2. Sample the transition time $T \in \text{Exp}(\sum_j \lambda_{i,j})$

Using the discrimination sampling method makes transitioning the state an $\sim \mathcal{O}(1)$ computation on average, assuming intensities of similar magnitudes. Uniformly sampling from an array of cumulative intensities can also be done, with worst case complexity $\sim \mathcal{O}(\ln(nm))$ if one uses an efficient search algorithm. The latter requires constructing arrays of cumulative intensity, implying a one-time additional complexity of $\sim \mathcal{O}(nm)$, along with additional memory footprint.

2.3 Reference price

In Section 2.2, we briefly mentioned the need for a method of valuing strategies. A method often used in the literature is to define some terminal time T , or a similar condition, where the agent is forced to execute a disadvantageous trade if it has not yet accomplished its goal. For example, in a MDP in [2] the agent is forced to purchase a share via market order if the bid i_B reaches some predetermined limit. Similarly, in [3], the agent was forced to execute the remaining inventory via market order if the time reaches a limit T . The MDP is then stated as a cost minimisation problem, and the optimal policy minimises the expected purchase price.

In our experiments, we benchmark all trades against some future price $R_{\delta t}$. This allows us to value an arbitrary set of trades.

Observe a strategy $\alpha(s, t) : S \rightarrow A(s)$ which we have executed on some market until the strategy terminates. At termination, it has bought n shares at prices $\{b_i\}$ and sold n shares at prices $\{s_i\}$. In this case, it is simple to calculate the total profit/loss by summing the pairwise differences between the buy and sell prices, even if it is not in their chronological order:

$$P = \sum_i s_i - \sum_i b_i = \sum_i (s_i - b_i). \quad (2.33)$$

This method becomes ill defined when $n_s \neq n_b$. The prices $\{b_i\}$, $\{s_i\}$ may no longer be relevant, and even if our trades have recently executed, there is no

guarantee that we could execute the opposite trade(s) at the same price(s). Our solution is a reference price R to benchmark the prices against. This redefines the profit/loss as:

$$P = \sum_{i=1}^{n_s} (s_i - R) + \sum_{j=1}^{n_b} (R - b_j). \quad (2.34)$$

We see that if $n_s = n_b$, we precisely recover Equation (2.33). The reference price represents what we can expect the shares to trade for in the future. In the work, we model this as being the price some time δt after termination. The price δt after termination can be obtained by waiting time δt , and sampling a price $R = R_{\delta t}$ according to some scheme. The model we used for the reference price is the middle of the spread:

$$R_{\delta t} = \frac{i_A + i_B}{2}. \quad (2.35)$$

Since we calculate the reference price after termination, it enters the value iteration equations in the terminal value v_T . In terminal state $s \in \partial S$, the value of our policy is

$$V(s, \alpha) = \mathbf{E}[v_T(s)]. \quad (2.36)$$

The value $v_T(s)$ is the profit/loss of Equation (2.34)

$$\mathbf{E}[v_T(s)] = \mathbf{E}\left[\sum_{i=1}^{n_s} (s_i - R_{\delta t}) + \sum_{j=1}^{n_b} (R_{\delta t} - b_j)\right] = (n_b - n_s)\mathbf{E}[R_{\delta t}|s] + \sum_{i=1}^{n_s} s_i - \sum_{j=1}^{n_b} b_j. \quad (2.37)$$

To find an optimal strategy through value iteration, we need to calculate the expected reference price, given $s \in \partial S$:

$$\mathbf{E}[R_{\delta t}|s]. \quad (2.38)$$

Having access to the structure of Section 2.2, this can comfortably be accomplished using Monte-Carlo methods if no analytic solution is possible. However, we also note that choosing $\delta t = 0$ makes $R_{\delta t} = R_0$ a deterministic function of the terminal state $s \in \partial S$. This choice is motivated by the fact that the reference price $R_{\delta t}$ affects the optimal strategy only through its expected value in the terminal reward, $v_T(s)$.

Before moving on, we observe how the reference price behaves in some special cases. First, managing to execute a limit order leads to a profit of half a spread

in all cases where there are additional orders behind the agent order. For example

$$0, \begin{bmatrix} 0 & 0 \\ -1 & -1 \\ 1 & 0 \end{bmatrix} \rightarrow -e_1, \begin{bmatrix} 0 & 0 \\ -1 & 0 \\ 1 & 0 \end{bmatrix}, \quad (2.39)$$

where the transition is caused by a unit size market order to the bid. Here, the value at termination would be precisely half the spread

$$v_T = R_0 - 1 = \frac{1+2}{2} - 1 = \frac{1}{2}. \quad (2.40)$$

However, if the agent order is the only order at the price level and a market order of unit size arrives

$$0, \begin{bmatrix} 0 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix} \rightarrow -e_1, \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad (2.41)$$

then the order causes the price to move to the agents disadvantage. The result is a net loss:

$$v_T = \frac{2-1}{2} - 1 = -\frac{1}{2}. \quad (2.42)$$

Inversely, if the agent places a market order at the ask for a single share when the depth is larger than one

$$0, \begin{bmatrix} 2 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \rightarrow -e_0, \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad (2.43)$$

then this causes a loss

$$v_T = \frac{-1+0}{2} - 0 = -\frac{1}{2}. \quad (2.44)$$

If the depth is only a single unit, the agents order causes a price change to its advantage

$$0, \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \rightarrow -e_0, \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}. \quad (2.45)$$

The above example in this case yielding a reward of

$$v_T = \frac{3-1}{2} - 0 = 1. \quad (2.46)$$

These behaviours have probabilistic analogues for $\delta t > 0$, but are deterministic in the $\delta t = 0$ case.

2.4 Historical Data

Historical and live data for limit order books is generally provided in tiers, called *levels*. The most basic tier is *level 1* data, and is what one is likely to encounter when placing orders as a retail investor. Crucially, it only contains the best bid and ask prices, and no information about price levels below/above. With higher tiers, the depth at some number of price levels below/above the bid/ask are provided as well [10].

Since it is the most readily available, we use level 1 data for historical evaluations. Having only two price levels also results in a tractable state space. Level 1 data requires some adaption of the model, in Section 2.1. For example, the matrix state model of Section 2.1 presumes detailed knowledge of all price levels. In Appendix B, we give the algorithm used for making the historic data compatible with the modified model of Section 2.4.1.

There are mathematical methods for dealing with MDPs under limited information. For example, a Partially Observable Markov Decision Process (POMDP) treats the observable state s as only containing partial information about the true state $\bar{s}$. This is directly applicable to our situation: We can only observe the bid and ask, not the price levels above or below. Reframing the optimal order placement problem as a POMPD would not necessarily increase the real-world applicability of the Markov model approach. This is because other actors have access to data of a higher level, leading to a disadvantageous information asymmetry. Rather, it would be more fruitful to obtain a higher level of data.

2.4.1 Reduced state representation

Since we can only observe information regarding the bid and the ask, we rewrite our state representation as

$$s = ([d_B, d_A, \Delta], \theta), \quad (2.47)$$

where d_B, d_A are the depths at the bid and ask respectively, and Δ is the spread. Since we are evaluating strategies by comparing prices to the instantaneous

reference price R_0 , we can save on state-space by not keeping track of the absolute price, e.g by having the bid price as part of the state. The removal of any absolute price means that we can no longer use price-based termination conditions.

The variable θ is some representation of the agents orders. When the agent has no orders, we adopt the convention

$$\theta = 0 \quad (2.48)$$

regardless of the specific representation of θ .

We want to keep the model fully within an observable Markov process framework, and not introduce any additional modelling of variables we can not observe directly. Therefore, whenever the bid (ask) is emptied, we will assume that the number of empty price levels $\delta\Delta \geq 0$ below (above) and the depth $d > 0$ at the first non-empty price level are both random variables independent of earlier states. Data shows* that a geometric distribution for $\delta\Delta$ is a good fit:

$$\mathbf{P}(\delta = i) \sim (1 - p_\delta)^{i-1} p_\delta, \quad (2.49)$$

where p_δ is a parameter to be fit to data. The depth at the new price level does not clearly obey some distribution, and varies day-by-day and even during intraday intervals. We model that the probability for some depth d is proportional to the number of times the value has been observed in a historical sample

$$\mathbf{P}(d = n) \propto |\{s \rightarrow s' : d = n\}|, \quad (2.50)$$

where we truncate the distribution at some D corresponding to the max depth of our state space. If one needs to sample d , discrimination sampling can be used to do so in $\mathcal{O}(1)$ time.

The intensities described in Section 2.1 remain the same with the caveat that we no longer model transitions due to orders below (above) the bid (ask). As a result, we can not permit the agent to have *deep* orders below the bid or above the ask. Say the agent has some order $\Delta_\theta < \Delta$ above the bid. If a limit buy order of size k arrives at $\Delta > \Delta' > \Delta_\theta$, then we immediately cancel the agent

*See Appendix B.4

order as part of the transition

$$([d_B, d_A, \Delta], \theta) \rightarrow ([k, d_A, \Delta - \Delta'], 0). \quad (2.51)$$

As we do not model any time delays between the agent and the order book, this is equivalent with forcing the agent to cancel its order if it becomes deep, with the added benefit of saving on state space since we do not need to index the intermediary state

$$s = ([k, d_A, \Delta - \Delta'], \theta). \quad (2.52)$$

This state space representation can be extended to a case where one has access to more price levels than just the best bid and ask. If one wants to include the price below and above the bid and ask, a representation such as

$$([d_{B-1}, \Delta_{-1}, d_B, \Delta_0, d_A, \Delta_1, d_{A+1}], \theta), \quad (2.53)$$

is a natural extension Equation (2.47). If more price levels are needed, pairs $(d_{A/B \pm i}, \Delta_{\pm i})$ can be appended to the state.

Chapter 3

Optimal Order Placement on Synthetic Data Using the Matrix State Representation

We study the execution of either a buy or sell order before the price moves out of the price range represented in the state space S . The states $s \in S$ are represented using the matrix representation of Section 2.1.2. We use synthetic data generated using the dynamics of Section 2.1 to evaluate and study the optimal strategy under ideal circumstances.

3.1 Experimental setup

3.1.1 Decision process on the matrix state representation

Consider a scenario where we trade some range of n price levels relative to a price p_0 . Our objective is to execute a trade of unit size before the price moves out of this range. By Equation (2.34), the reward of executing a trade at price i is

$$R = \pm(R_0 - i), \quad (3.1)$$

where R_0 is the instantaneous reference price and the sign is positive for a purchase. We define the price moving out of the range as either the ask reaching its minimum possible value, $i_A = 0$, or the bid reaching its maximum possible value, $i_B = n - 1$. Should this occur, the episode is terminated and we receive zero reward. The terminal states are states where we have either

executed a trade, or have been priced out:

$$\partial S = \{s : i_A = 0\} \cup \{s : i_B = n - 1\} \cup \{s : |p| > 0\}. \quad (3.2)$$

We can express the termination reward $v_T(s)$ for $s \in \partial S$ as

$$v_T(s) = \begin{cases} R, & s \in \{s : |p| > 0\} \\ 0 & \text{otherwise} \end{cases}, \quad (3.3)$$

where R is as in Equation (3.1).

The order book has $n = 4$ price levels, and we allow a maximum depth of $d_{max} = 3$ limit orders at each price level. We restrict the agent to a single limit order in the order book at a time. When the agent places a limit order, it must wait for at least one non-agent to arrive order before cancelling*. With these hyper-parameters and restrictions, the state space size is

$$|S| = 15744, |\partial S| = 11072. \quad (3.4)$$

For the model parameters, we set all linear intensity parameters to unity:

$$\lambda_b^a = 1, \quad a \in \{\mathbf{B}, \mathbf{S}\}, \quad b \in \{\mathbf{M}, \mathbf{L}, \mathbf{C}\}. \quad (3.5)$$

The distributions for the size of limit and market orders is modelled as being exponential, as in Equation (2.17). For the distribution parameters we choose

$$\alpha_{\mathbf{M}} = \alpha_{\mathbf{L}} = 2. \quad (3.6)$$

For both order types, this gives us

$$p_1 \sim 0.86, \quad p_2 \sim 0.12. \quad (3.7)$$

We truncate the maximum order size to $k_{max} = 2$. With this truncation, the occupancy rate of the jump matrix for waiting, $P_{s,s'}(a_W)$, is $\sim 6.29 \cdot 10^{-4}$.

*This restriction is to ensure that all strategies a.s terminate in finite time. If we had no such restriction on limit orders, a strategy which repeatedly places a limit order and then immediately cancels it would never terminate.

3.1.1.1 Transitions resulting from agent orders

Market orders and cancellations result in deterministic transitions. If $a \in A(s)$ is a market order or cancellation in state s , then

$$P_{s,s'}(a) = 1 \text{ for some } s' \in S. \quad (3.8)$$

As we force the agent to wait at least one transition after placing a limit order, the action of placing a limit order does not result in a deterministic transition. We denote a deterministic limit order action as a_L , i.e an action with a transition matrix as in Equation (3.8), and a wait action as a_W . Placing a limit order and then waiting can then be viewed as the compounded action $a_{LW} = a_W a_L$, with transition matrix elements

$$P_{s,s'}(a_{LW}) = P_{s,s''}(a_L)P_{s'',s'}(a_W). \quad (3.9)$$

The reward for placing a limit order, a_{LW} , then becomes

$$R_{s,s'}(a_{LW}) = R_{s,s''}(a_L)R_{s'',s'}(a_W) = I\{s' \in \partial S\}v_T(s'). \quad (3.10)$$

As $a_W \in A(s)$ for all $s \notin \partial S$, and

$$P_{s,s'}(a_L) = 0, \forall s' \in \partial S, \quad (3.11)$$

the compounded action as constructed above is well defined. This compounded action $a_W a_L$ is the action available to the agent in lieu of a_L , in the appropriate states.

3.2 Results

3.2.1 Observed behaviour of the optimal strategy

Having defined the MDP in Section 3.1.1, we can obtain an approximation for the optimal strategy using value iteration. Studying the resulting strategy α^* , we find that it exploits the properties of the instantaneous reference price R_0 discussed in the latter part of Section 2.3. The strategy seeks to have a limit order executed such that there is depth behind the agent order and the spread

is large. For example, in a order book state* such as

$$x = \begin{bmatrix} -3 & 0 \\ -1 & -1 \\ 0 & 0 \\ 2 & 0 \end{bmatrix}, \quad (3.12)$$

the agent will choose to wait, with the optimal outcome being a market order of unit size. This would result in a terminal reward of

$$v_T(s) = \frac{1+3}{2} - 1 = 1. \quad (3.13)$$

An order book state such as

$$x = \begin{bmatrix} -1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 2 & 0 \end{bmatrix} \quad (3.14)$$

will cause the agent to cancel its order, as a unit size market order would lead to a net-zero terminal reward

$$v_T(s) = \frac{3+1}{2} - 2 = 0, \quad (3.15)$$

and an order of size two would lead to a net loss

$$v_T(s) = \frac{2+1}{2} - 2 = -0.5. \quad (3.16)$$

The agent engages in *greedy* behaviour, where it will avoid exposure to profitable termination when the spread is small. In the order book state

$$x = \begin{bmatrix} 0 & 0 \\ -2 & 0 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad (3.17)$$

*Here, we say "order book state x " to distinguish from the full state $s = (p, x)$. For this experiment, the position vector, p , can not be discarded. It is omitted from the notation for simplicity as it is zero in all non terminal states.

the agent will cancel its limit order. If it would instead choose to wait, and a market order of unit size arrives, then the terminal reward would be

$$v_T(s) = 2 - \frac{1+2}{2} = 0.5. \quad (3.18)$$

A market order of size two would result in a net zero profit. The expected reward is less than the reward for the same event under the larger spread of Equations (3.12) and (3.13). Finally, the agent will only place a market order in six states, all being akin to

$$x = \begin{bmatrix} -1 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad (3.19)$$

where the order would be placed at the ask. The reason for this is that the market impact of the order causes the reference price to increase, in the above resulting in a terminal reward of

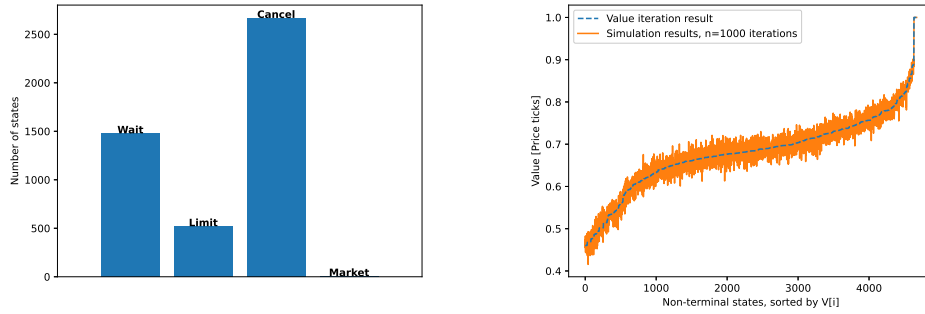
$$v_T(s) = \frac{0+4}{2} - 1 = 1. \quad (3.20)$$

In Figure 3.1a, we see the distribution of action types for states $s \notin \partial S$. The expected MDP rewards are shown in Figure 3.1b.

We see that a small number of states have a deterministic value of $V(s_i, \alpha) = 1$. These are precisely the states outlined in and around Equation (3.19), or states where the agent can navigate to such a state in a deterministic manner. For example, by cancelling some limit order

$$\begin{bmatrix} 0 & -2 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} -1 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \quad (3.21)$$

and then immediately placing the market order at price $i = 1$.



(a) Histogram over action type for non-terminal states. Six states result in a market order, corresponding to states where the market order moves the reference price, R_0 , so that the trade is profitable.

(b) Results for $n = 1000$ iterations per initial state. The states have been sorted in ascending order of $V(s_i, \alpha)$.

Figure 3.1: Results for synthetic data, using the matrix state representation.

Chapter 4

Optimal Order Placement on Synthetic and Historic Data Using the Reduced State Representation

In this section, we study the optimal strategy for the purchase of a single share on a state space S represented by the reduced state of Section 2.4.1. We formulate a MDP to be used both on synthetic data and historic data. This allows us to directly compare the ideal results on synthetic data with the results on historic data.

4.1 Experimental setup

4.1.1 Decision process on the reduced state representation

Similarly to the previous MDP of Section 3.1.1, we task our agent with executing a single trade. This time, we are only focused on the purchase of a share and do not allow the agent to place any sell orders. In the state representation

$$s = ([d_B, d_A, \Delta], \theta), \quad (4.1)$$

we can represent the agent order as

$$\theta = [\delta, \Delta_\theta]. \quad (4.2)$$

The variable δ is the number of orders ahead of the agent, including itself. If $\delta = 0$, then the order has executed. The second variable, Δ_θ , is price of the agent order relative the current bid. If the agent has no order, then we will denote this $\theta = 0^*$. Having established the representation, we can write the set of terminal states as

$$\partial S = \{s : \theta \neq 0, \delta = 0\}. \quad (4.3)$$

As in Section 3.1.1, we force the agent to wait after placing a limit order. In addition, we force the agent to wait after cancelling a limit order[†]. The construction of the cancel-then-wait action $a_{CW} = a_W a_C$ is analogue to the construction of a_{LW} in Section 3.1.1.1.

As mentioned in Section 2.4.1, the reduced state representation does not allow for a termination condition dependent on absolute price. If we were to completely omit an incentive to execute an order, then the optimal strategy would be to wait for the most profitable state(s) $s \in S$ before placing any orders. This is not an attractive proposition, as such a state may never be encountered when trading on a real exchange.

We resolve this by introducing a penalty $c > 0$ for each wait action, a_W , the agent chooses to take. This penalty enters the reward matrix:

$$R_{s,s'}(a_W) = -c + I\{s' \in \partial S\}v_T(s'). \quad (4.4)$$

The penalty acts as a control variable for the execution time of the strategy. Consider the following: If we approximate that the spread at execution is always zero, $\Delta = 0$, then the profit for a limit order executing would be half a tick, and the loss for placing a market order would be half a tick. The difference between the two is one full tick. If have a limit order in the order book, and we expect it to execute in an average of N transitions, then we would prefer to wait as opposed to placing a market order if

$$N < \frac{1}{c}. \quad (4.5)$$

*If one wants to formulate a MDP which both allows buy and sell orders, a viable representation is $[\delta, \Delta_\theta, \ell]$, where $\ell \in \{-1, 1\}$ is the polarity. In a code implementation, the variable ℓ can then also be used to determine order existence, with $\ell = 0$ representing no order.

[†]This is to prevent certain behaviours resulting from the penalty only being applied for a wait action a_W .

We can expect the optimal strategy to converge towards placing a market order at $t = 0$, as c is increased.

4.1.2 Evaluation metrics

Because we introduced the penalty, c , to the MDP reward, we can no longer interpret the value to be in units of *currency*. We define the cost of trading c_T in $s' \in \partial S$ to be the return:

$$c_T = \mathbf{E}\left[\frac{\pm(R - p)}{R} \mid s \in \partial S\right], \quad (4.6)$$

where p is the purchase price, and R is the reference price in absolute units of currency. The difference between the absolute reference price, R , and the purchase price p is precisely our terminal reward $v_T(s)$ times the tick size δp . Further, we approximate the absolute reference price R in the denominator by the volume weighted average price $\bar{R}$ during the day the episode is sampled from. Then we can write the cost of trading as

$$c_T = \frac{\delta p}{\bar{R}} \mathbf{E}[v_T(s)]. \quad (4.7)$$

As a measure for execution time T , we use the trade rate r

$$r = \mathbf{E}\left[\frac{1}{T}\right]. \quad (4.8)$$

This has some similarity to the *participation rate*, which is the proportion of trades occurring which the agent is one of the parties in. The trade rate r is easier to define in terms of the MDP, and is chosen for expediency.

We created two simple stationary strategies as a benchmark for comparison: *Bid Plus One* and *Market Order*. The details of these are given in Appendix C.

4.1.3 State space, data sequences and initial states

We have used the methods of Appendix B to make the data compatible with the framework of Section 2.2 and state space of Section 2.4.1. The data used is detailed in Appendix B.4.

Hyper-parameters: For state space hyper parameters, we choose a maximum

depth of $d_{max} = 8$ and a maximum spread of $\Delta_{max} = 2$ for a state space size of

$$|S| = 2592, |\partial S| = 1152. \quad (4.9)$$

The state space hyper parameters have been chosen through consideration of the trade-off between data admissibility and computational expedience. For the transition hyper parameters, we use $(\delta\Delta)_{max} = 0$, $k_{max} = 4$ after considering the distributions observed. Plots of the typical distributions are shown in Appendix B.4.

Historic episodes: Each episode consists of a chronological sequence $E_k = \{e_i\}_{n_k \geq i \geq 1}$ of events e_i . These events are defined in Appendix B.1. An episode E_k is disjoint from other episodes. For all events e_i , the incoming and outgoing states $s, s' \in S$. Per sequence, the strategies always start on the same state, the incoming state of e_{n+1} for a fixed $n = 200$. For the optimal strategy, the model parameters are estimated on the sub-sequence $\{e_i\}_{1 \leq i \leq n}$ prior to the starting state e_{n+1} .

Synthetic episodes: For each sequence E_k , we take the corresponding initial state, s_k together with the estimated parameters, Λ_k , and then generate synthetic episodes by sampling the transitions according to our model dynamics.

For each pair of initial state and parameters, (s_k, Λ_k) , and penalty, c , we sample the outcome of the optimal strategy $n_{iter} = 300$ times. Per penalty, we then average these outcomes to obtain estimates for the rate, r , and cost of trading, c_T , as a functions of the penalty. The expected MDP reward need not be estimated using Monte-Carlo methods. It is precisely $V(s_k, \alpha^*)$ on synthetic data generated using the dynamics the strategy is optimal for.

4.2 Results

4.2.1 Results on synthetic data

The results of the simulations are shown in Figure 4.1.

In Figure 4.1a, we see the expected Markov reward for a range of penalties $c \in [0.01, 0.35]$. The optimal strategy, α^* , by definition fulfills $V(s, \alpha^*) \geq V(s, \alpha)$ for any other strategy, α . As a result, we expect the optimal strategy to

outperform the simple strategies in terms of MDP reward on synthetic data. As the penalty increases, the optimal strategy converges towards being a market order strategy. This is in-line with the thought experiment of Section 4.1.1. Figure 4.1b show the rate r as a function of the penalty c . A rate of

$$r = 1 \tag{4.10}$$

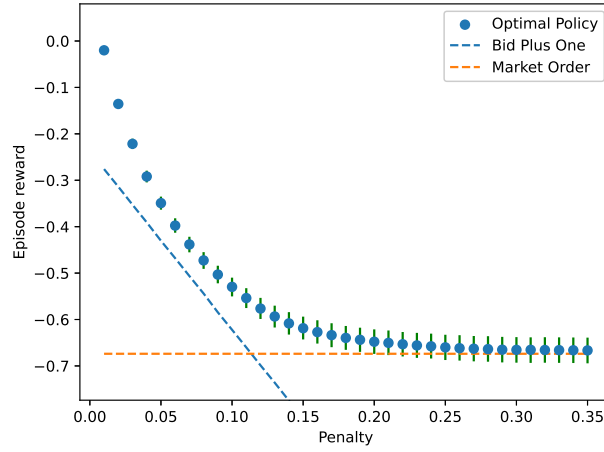
implies a pure market order strategy.

Finally, Figure 4.1c shows the cost of trading, c_T , as a function of the rate, r . Here, we see that the optimal strategy offers a better cost of trading than the Market Order strategy for each rate r . The Bid Plus One strategy lands along the line of optimal strategies, but has the disadvantage that the rate can not be varied.

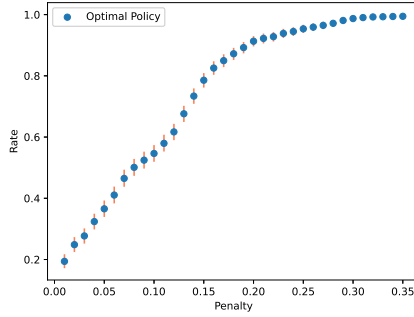
4.2.2 Results on historic data

The results on historic data are shown in Figure 4.2. We note that the optimal strategy no longer outperforms the Market Order strategy in terms of MDP rewards for all rates r . However, the optimal strategy maintains a better cost of trading for each r . The Bid Plus One strategy performs similarly to the synthetic case, relative to the optimal strategy. In Figure 4.3, we directly compare the synthetic and historic results.

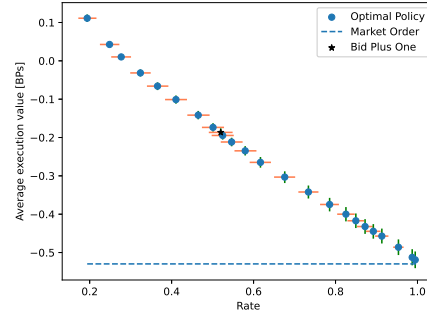
The Market Order strategy had identical performance on historic and synthetic data. Since the Market Order strategy places a market order at $t = 0$, its value is a deterministic function of the initial state. The initial state and parameter pair (s_k, Λ_k) was used to generate the synthetic episodes, leading to the same results.



(a) The MDP reward for the strategies. At $c = 0.35$, the optimal strategy places a market order in $\sim 99\%$ of the initial states.

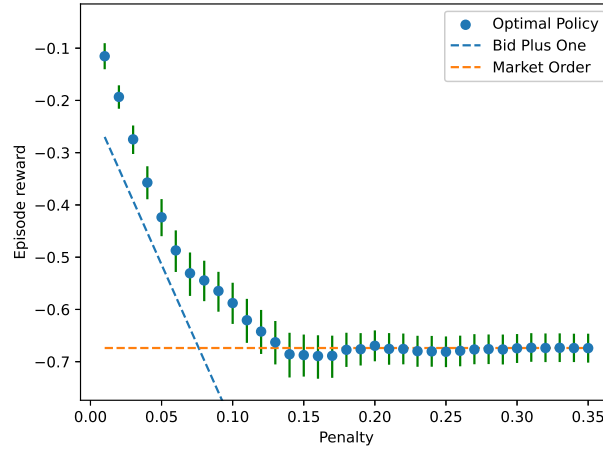


(b) The rate r of the optimal strategy as a function of the penalty c .

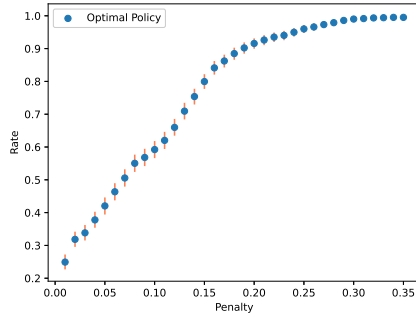


(c) The cost of trading c_T as a function of the rate r . Bid Plus One becomes a single point and Market Order a constant line, see Appendix C.

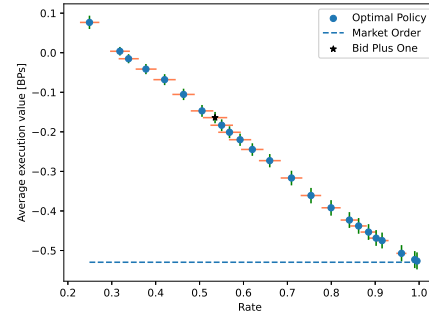
Figure 4.1: Results on synthetic data.



(a) The MDP reward as a function of the penalty c . The optimal strategy outperforms the Bid Plus One strategy, but now is within the margin of error from the Market Order strategy for larger penalties, c .

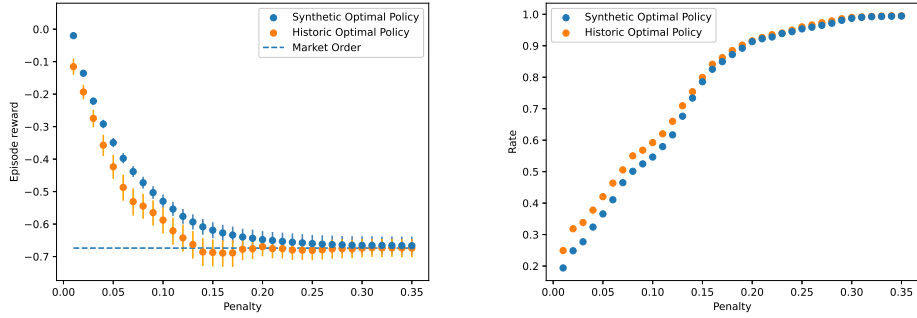


(b) The rate of the optimal strategy, as a function of the penalty.



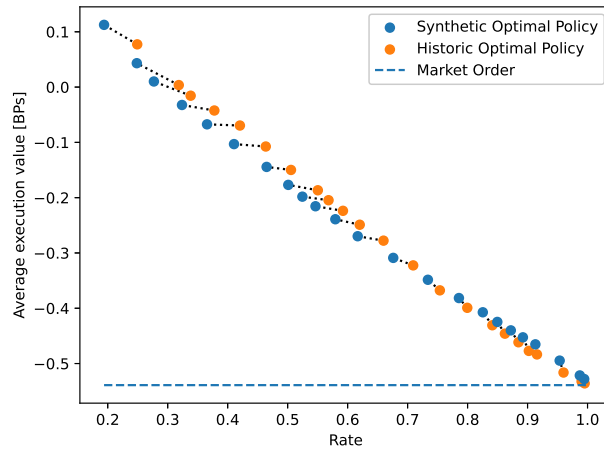
(c) The cost of trading as a function of the rate, r .

Figure 4.2: Results on historic data.



(a) Because the historic data is not generated according to the model dynamics, we can not expect the MDP rewards to be $V(s, \alpha^*)$ for each s .

(b) Rate as a function of penalty.



(c) Cost of trading as a function of rate. Black dots connect synthetic and historic data points corresponding to the same penalty c .

Figure 4.3: Comparison between synthetic and historic results.

4.2.3 Discussion

Since historic data is not generated according to the model dynamics, we can not expect the optimal strategy to have the expected reward $V(s, \alpha^*)$ in all states. However, the comparison in Figure 4.3 imply that the model does capture some aspects of the limit order books behaviour. The optimal strategy still outperforms the Bid Plus One strategy in terms of the MDP, and is within the margin of error of the Market Order strategy.

As we see in Figure 4.3a, the historic evaluation lead to a lower reward for all penalties. Two contributing factors could be that the strategy overestimates the value of its eventual execution, and underestimates the time to execution.

In Figure 4.3c, we see that the execution value is indeed overestimated per penalty, c . In Figure 4.3b, we see that the optimal strategy has a higher trade rate on historic data. The higher trade rate could imply that we *overestimate* the time to execution. This could be the result of the strategy underestimating the probability that we encounter a state where the estimated optimal action is to place a market order. Placing more market orders than expected would explain both the increased trade rate and decreased execution value: Market orders terminate the episode instantly, and generally lead to worse execution values compared to limit orders.

There is no guarantee that the optimal strategy should perform *worse* in terms of the MDP on historic data. If the model were to systematically underestimate the execution value, then this would be to the optimal strategy's advantage when evaluating on historic data.

Chapter 5

Conclusions and Future work

In Section 4, we applied the optimal Markov strategy method to historic data, and compared this to the results on synthetic data. The optimal strategy resulted in a better cost of trading c_T compared to the Market Order strategy as a function of the rate r . In terms of the MDP, the optimal strategy performed worse on historic data when compared to synthetic data. It still outperformed the Bid Plus One strategy for all penalties, but was within the margin of error from the Market Order strategy for penalties $c > 0.15$.

We used the instantaneous reference price R_0 to benchmark the execution prices against. This was done for expediency, and resulted in the optimal strategy exploiting short term, deterministic properties of the reference price, as seen in Section 3. Investigating the use of a reference price given by a future mid price would put the model under more scrutiny, as the reference price would depend on a model estimation for a distribution at some point in the future.

In terms of the method, the model of Section 2 is interchangeable with any Markov model we can formulate a MDP for optimal order placement on. It is possible that a more sophisticated model can be proposed, and that this model better describes the micro-structure behaviour of the LOB. When attempting such a model, we must consider the tractability of the state space S . For instance, introducing additional price levels as in Equation (2.53) results in exponential growth of the state space. Calculating the jump matrix for the intensities in Section 2.1 is a $\mathcal{O}(|S|)$ operation.

The actions available to the agent can be extended to also model latency

experienced when trading on an exchange. This can be achieved using the MDP framework. As an illustrative toy model, assume that when we place a limit order there is a $0 \leq \gamma \leq 1$ chance that some other order arrives to the exchange before the agents. The transition matrix, $P_{s,s'}(a_L^\gamma)$ for the latency action a_L^γ would then be

$$P_{s,s'}(a_L^\gamma) = \gamma P_{s,s''}(a_W) P_{s'',s'}(a_L) + (1 - \gamma) P_{s,s''}(a_L) \quad (5.1)$$

where a_W, a_L are the zero latency actions used in the work*.

If we are interested in studying related problems such as the optimal order execution problem[1], or we want to find optimal day trading strategies, then we can do so by formulating the MDP accordingly. For example, we can find a day trading strategy by forcing the agent to have purchased and then sold a share prior to termination.

It would be of interest to directly compare the optimal Markov strategy to reinforcement learning methods. For a MDP, a reinforcement learning algorithm also approximates the α^* maximising $V(s, \alpha)$. A tuned tabular Q-learning algorithm executed on synthetic data would converge to precisely the same optimal policy α^* as in the Markov case. However, such an algorithm would not be limited by the assumption of some dynamics for the LOB when trained on historic data. Deep Q-learning [11], where the policy function $\alpha(s)$ is a neural network

$$\alpha(s) = \arg \max_{a \in A(s)} Q(s, a, \theta), \quad (5.2)$$

has a fixed memory footprint dependent on network size. Thus, it is possible that the size of the state space can be increased, by adding more price levels to the state representation. Using a model such as a *Long Short-Term Memory* neural network [12] would allow the network to remember transitions for some time, enabling the model to discern the current market dynamics, analogue to the parameters of the model of Section 2.1.

*Here, we have brazenly assumed that $a_L \in A(s'')$ in the first term of the r.h.s. We can numerically handle the cases where $a_L \notin A(s'')$ by putting $P_{s'',s'}(a_L) = 1$.

References

- [1] G. Xin, “Optimal placement in a limit order book.” doi: 10.1287/educ.2013.0113. [Online]. Available: <https://doi.org/10.1287/educ.2013.0113> [Pages 1 and 44.]
- [2] H. Hult and J. Kiessling, “Algorithmic trading with markov chains,” 2010. [Pages 1, 9, 11, 14, 16, and 21.]
- [3] Y. Nevmyvaka, Y. Feng, and M. Kearns, “Reinforcement learning for optimized trade execution,” in *Proceedings of the 23rd International Conference on Machine Learning*, ser. ICML '06. New York, NY, USA: Association for Computing Machinery, 2006. doi: 10.1145/1143844.1143929. ISBN 1595933832 p. 673–680. [Online]. Available: <https://doi.org/10.1145/1143844.1143929> [Pages 1 and 21.]
- [4] H. Hultin, H. Hult, A. Proutiere, S. Samama, and A. Tarighati, “Reinforcement learning for optimal execution in high resolution markovian limit order book models,” qC 20210531. [Online]. Available: <http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-295423> [Page 1.]
- [5] ———, “A generative model of a limit order book using recurrent neural networks,” qC 20210531. [Online]. Available: <http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-295414> [Page 1.]
- [6] D. Bertsimas and A. W. Lo, “Optimal control of execution costs,” *Journal of Financial Markets*, vol. 1, no. 1, pp. 1–50, 1998. doi: [https://doi.org/10.1016/S1386-4181\(97\)00012-8](https://doi.org/10.1016/S1386-4181(97)00012-8). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1386418197000128> [Page 4.]
- [7] D. Byrd, M. Hybinette, and T. H. Balch, “Abides: Towards high-fidelity multi-agent market simulation,” in *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*,

- ser. SIGSIM-PADS '20. New York, NY, USA: Association for Computing Machinery, 2020. doi: 10.1145/3384441.3395986. ISBN 9781450375924 p. 11–22. [Online]. Available: <https://doi.org/10.1145/3384441.3395986> [Page 4.]
- [8] J. Norris, *Markov Chains*, ser. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1998. ISBN 9780521633963. [Online]. Available: <https://books.google.se/books?id=qM65VRmOJZAC> [Pages 4, 6, and 9.]
- [9] A. G. Sutton, R. S. Barto, *Reinforcement learning: An introduction*. MIT Press, 2018. [Page 19.]
- [10] “Level 1: Definition, How Trading Screen Works, and Accessibility, howpublished = <https://www.investopedia.com/terms/l/level1.asp>, note = Accessed: 2023-05-10.” [Page 24.]
- [11] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing atari with deep reinforcement learning,” 2013. [Page 44.]
- [12] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735 [Page 44.]
- [13] “Stock api documentation,” <https://polygon.io/docs/stocks>, accessed: 2023-04-21. [Pages 50 and 57.]

Appendix A

Markov Processes

This section briefly goes over aspects of Markov processes related to the work omitted in the introductory Section [1.2](#).

A.1 Potentials

Suppose we have a discrete jump process $s(n)$, $n \geq 0$ on a countable state space S . The process hits some set of terminal states ∂S after T transitions such that

$$\mathbf{P}(T < \infty) = 1. \quad (\text{A.1})$$

If we define the agent to receive reward R_{s_i, s_j} for each transition $s_i \rightarrow s_j$ then, starting at state $s(0)$, the total reward is

$$R = \sum_{n=0}^{T-1} R_{s(n), s(n+1)} + v_T(s(T)), \quad (\text{A.2})$$

where $v_T(s \in S)$ is some terminal reward function. We define the potential $\phi(s)$ in state s to be the expected reward of the process which starts at $s(0) = s$. This can be written

$$\phi(s) = \mathbf{E}\left[\sum_{n=0}^{T-1} R_{s(n), s(n+1)} + v_T(s(T)) \mid s(0) = s\right], \quad (\text{A.3})$$

where the termination time T and future states $s(n \geq 1)$ are random variables. Due to the Markov property, we can rewrite Equation [\(A.3\)](#) as

$$\phi(s) = \mathbf{E}[R_{s, s'} + \phi(s') \mid s(0) = s]. \quad (\text{A.4})$$

The moniker *potential* comes from the superficial similarity to the path-independent potentials of physics. Indeed, the potential in state s is independent of how the state arrived at s .

Potentials provide the framework with which we can value policies. If we have a MDP on $s(t)$ with a stationary policy $\alpha(s)$, then we can value the policy starting in state s as

$$V(s, \alpha) = \mathbf{E}[R_{s,s'}(\alpha(s)) + V(s', \alpha)]. \quad (\text{A.5})$$

The value can directly be interpreted as the potential in state s given the policy α .

A.2 State Indexation

When dealing with our state space S , we will often refer to states by some index i , i.e

$$s_i \in S. \quad (\text{A.6})$$

For example, we observe the jump matrix P , where $P_{i,j}$ is the probability for the next transition being

$$s_i \rightarrow s_j, \quad (\text{A.7})$$

given that we find ourselves in state s_i . To be able to calculate $P_{i,j}$ we must know *which* state the indices i and j refer to. Thus, we need to define a map from some range of positive integers J_n to our state space S ,

$$f : J_n \rightarrow S. \quad (\text{A.8})$$

This map is bijective, and its existence is a prerequisite for the set S being countable. Its inverse,

$$f^{-1} : S \rightarrow J_n, \quad (\text{A.9})$$

maps the states to their respective indices. In the work, this map is left implicit, but needs to be explicitly defined if one wants to implement a decision policy utilising the methods of this thesis. In-code, the map can be abstracted by an interface given in Listing A.1.

Listing A.1: IStateIndexer interface

```
class IStateIndexer:
    def to_index(self, x: np.ndarray) -> int:
```



```
raise NotImplementedError
```

```
def to_state(self, index: int) -> np.ndarray:
    raise NotImplementedError
```

If the state space is relatively small it is then expedient to store the indexation in memory through dictionaries. This allows for a few advantages

- Calculating $f(i)$ and $f^{-1}(s)$ are efficient $\mathcal{O}(1)$ operations, since they entail accessing an entry in a dictionary.
- It allows for 'brute-force' indexation, where one creates some range of states S' , such that $S \subseteq S'$ and then uses some auxiliary function g to determine which states belong to S .

The downsides of this approach become evident as the state space grows

- The memory footprint of the dictionaries grow as $\mathcal{O}(|S|)$.
- Instantiating the dictionaries is generally a $\mathcal{O}(|S|)$ operation.

Since the state space growth is exponential as a function of the various hyper-parameters, the dictionary approach used in this thesis quickly becomes unfeasible. One alternative is to figure out an algorithm for f . Presume we observe the state space

$$s_i = \begin{bmatrix} a_i \\ b_i \end{bmatrix}, \quad |a_i|, |b_i| \leq D. \quad (\text{A.10})$$

That is, two price levels with no room for agent orders. There are $(2D + 1)^2$ such states. We can index each of these through

$$s_n = \begin{bmatrix} n \bmod (2D + 1) \\ (n - (n \bmod (2D + 1)))/(2D + 1) \end{bmatrix}. \quad (\text{A.11})$$

This scheme generalises to more price levels, but becomes increasingly cumbersome to compute. This is very relevant since the indexation function f and the inverse f^{-1} are "bread-and-butter" components of the implementation, and are among the functions most often called.

Appendix B

Historical Data

B.1 Order Matching

As mentioned in Section 2.4, we use *Level 1* data to evaluate the strategies in Section 4.2.2. This data is not directly compatible with the state space representation outlined in Section 2.4.1. Further, the data does not contain any explicit information regarding what type of order(s) resulted in a change of quotes. To be able to use the data to evaluate strategies, we need to both translate the data into our state space representation format. The order types need to be deduced from the quotes and trades data. In this section, we outline an algorithm for *order matching*, which is used to obtain the results of Section 4.2.2.

The data is provided as two separate time series: quotes and trades. Quotes contain the best bid/ask, and the corresponding depths. Trades contain a price, and size, for trades that have occurred. For a fully detailed account of the data, we refer to the data source [13].

Exchanges: Multiple exchanges report the bid/ask simultaneously, and an integer *id* of the exchange reporting the current bid (ask) is given. The exchanges reporting the best bid and ask need not be the same. Trades also contain the id of the exchange that reported them, and are reported from every exchange, not just the exchange(s) providing the bid/ask.

Sizes and prices: The depth of the quotes is given in units of round *lots*, with the size of a lot varying between shares. For the \$AAPL data used, the lot size is one hundred shares. Trades are always given in units of singular

shares. Both time series give prices in the currency the stock is denoted in. Trades need not occur at the best bid or ask, but can occur at any price, inside or outside the spread. We use lots as the unit, i.e a depth at the bid of $d_B = 2$ represents two lots. If the agent places a market order of unit size, then this represents purchasing a single lot.

Timestamps: Various timestamps are given for both time series, corresponding to the different times the actors involved generated or received the data. All timestamps are given in units of nanoseconds. Some timestamp types may be missing for a quote or trade. Since it is guaranteed to exist and makes for a well defined comparison, we will use the timestamp of when the data arrived at the data-broker and fully disregard the other timestamps.

Duplicates: There may be duplicate sequential quotes reported at different timestamps. Going forward, we will disregard such data points to reduce the computational complexity. In the proposed order matching algorithm, this will not affect the final sequence of states.

Mismatched order books: On rare occasions the quote can imply a crossed order book, $i_B \geq i_A$, with equality being the most common. These states are as a rule resolved within milliseconds. That such a situation can occur is a result of the bid and ask being reported from two different exchanges.

B.1.1 A simple order matching algorithm

The underlying assumption of this algorithm is that every state is reported by the quotes. That is, there are no intermediary states between two quote which can not be deduced by observing the difference between the two. What we have are two time series $T_0 = \{\tau_{tn}\}$ and $Q_0 = \{q_{tm}\}$ of trades and quotes respectively. The trades τ_{tn} are tuples of time, exchange, price and size

$$\tau_{tn} = [t_n, \text{id}, p, n]. \quad (\text{B.1})$$

The quotes q_{tm} are tuples of time, exchanges, bid, ask and corresponding depths

$$q_{tm} = [t_m, \text{id}_B, \text{id}_A, p_B, p_A, d_B, d_A]. \quad (\text{B.2})$$

We have omitted some additional auxiliary fields which we do not use in the order matching algorithm. Before attempting to deduce the states and orders, we will remove entries from both time series by a sequence of *filters*, meant to

improve the quality of the final output.

First, we remove trades which we do not believe are relevant. Since trades are reported from all exchanges simultaneously, but the quotes only come from one or two, we will disregard all trades which do not originate from the exchange(s) currently reporting the bid/ask. Let t_m^n be the time of the latest quote for trade n

$$t_m^n = \max_{t_m} \{t_m : t_m < t_n\}. \quad (\text{B.3})$$

Then a trade τ_{tn} is included in the first reduced set $T_1 \subseteq T_0$ if

$$T_1 = \{\tau_{tn} \in T_0 : (id)_{tn} = (id_A)_{t_m^n} \vee (id)_{tn} = (id_B)_{t_m^n}\}. \quad (\text{B.4})$$

We also introduce some cutoff on trade size $n_{\min}$, and disregard trades smaller than this as insignificant. This results in the next reduced trade set T_2

$$T_2 = \{\tau_{tn} \in T_1 : (n)_{tn} \geq n_{\min}\}. \quad (\text{B.5})$$

The two operations described above result in most trades occurring at the bid or ask. Any remaining trades which do not occur at either have their price modified to the closer of the two. That is,

$$T_3 = \{\psi(\tau_{tn})\}, \quad (\text{B.6})$$

where

$$\psi(\tau_{tn}) = \begin{cases} \tau_{tn}, & \text{if } p_{tn} = (p_B)_{t_m^n} \vee p_{tn} = (p_A)_{t_m^n} \\ [\dots, (p_B)_{t_m^n}, \dots], & \text{if } |(p_B)_{t_m^n} - p_{tn}| \leq |(p_A)_{t_m^n} - p_{tn}| \\ [\dots, (p_A)_{t_m^n}, \dots], & \text{otherwise} \end{cases} \quad (\text{B.7})$$

Next, we turn our attention to the quotes. Any quote corresponding to a mismatched order book is replaced with a 'synthetic' trade at the price of the bid, with size $n = |d_A - d_B|$. This gives us

$$Q_1 = \{q_{tm} \in Q_0 : (p_B)_{tm} < (p_A)_{tm}\}, \quad (\text{B.8})$$

and

$$T_{\text{synthetic}} = \{[t_m, (id_B)_{tm}, (p_B)_{tm}, |(d_A)_{tm} - (d_B)_{tm}|] : q_{tm} \notin Q_1\}. \quad (\text{B.9})$$

We then add the synthetic trades to the non-synthetic

$$T_4 = T_3 \cup T_{\text{synthetic}}. \quad (\text{B.10})$$

Now that we have our final sets of quotes Q_1 and trades T_4 . We now want to parse this into a single sequence of *events*

$$e = [t, s, \theta, s'], \quad (\text{B.11})$$

where t is the timestamp, s is the incoming state, θ is the order causing the transition and s' is the outgoing state. Note that we require both θ and s' , since we must be able to differentiate cancellations from market orders, and receive information regarding empty price levels if the bid/ask is emptied.

Let $i \geq 0$ index $q_{tm} \in Q_1$ in chronological order. Initialise the algorithm with the state

$$s = [d_B, d_A, \Delta], \quad (\text{B.12})$$

implied by the first quote $q_0 \in Q_1$. For each pair of quotes q_{i-1}, q_i :

1. See what state variables have changed between $q_{i-1} \rightarrow q_i$.
2. Deduce what order(s) this implies. See if a trade has occurred at the relevant price within δt to differentiate cancellations and market orders.
3. Package these events into a sequence $\{[(t)_i, s_0, \theta_1, s_1], \dots, [(t)_i, s_{k-1}, \theta_k, s_k]\}$.
4. Append this sequence to some master sequence.

A few notes regarding this algorithm:

- For each i , the incoming state s_0 is always the state implied by q_{i-1} , and the final outgoing state s_k is always the state implied by q_i .
- Trade sizes are disregarded, and instead the depth differential is used to size market orders.
- A transition $(p_B, p_A) \rightarrow (p'_B, p_B)$ (or the opposite) is always treated as a market order in point 2, regardless of trades data
- Because of the assumption in the previous comment, any sequence in point 3 is at most two events long. If such a sequence is two elements long, then these are considered to have happened at the same time, yet in the order of the sequence. Further, because we have already discarded

all duplicate quotes, the sequence is never empty. If one extends the algorithm to *level 2* data, then the length of the sequence can exceed two entries.

B.2 'Synthetic Layer' on Historical Data

The orders of the agent need to be superimposed upon the historical data. Placing a limit order is straight-forward, as it 'slots in' to the state representation

$$([d_B, d_A, \Delta], 0) \rightarrow ([d_B, d_A, \Delta], \theta). \quad (\text{B.13})$$

However, future transitions now become more cumbersome as we need to track the position of the agent order relative to the bid. In addition, the agent order may impact the state itself when it is executed. The situation is somewhat simplified by having set the reference price R_0 to be instantaneous, since we do not need to propagate this impact over multiple transitions.

Here we will give a brief account of how we handle various situations when evaluating on historic data. As we are only concerned with singular limit buy orders, θ consists of

$$\theta = [\delta, \Delta_\theta, \ell], \quad (\text{B.14})$$

where δ is the position in the execution queue, $\Delta \geq \Delta_\theta \geq 0$ is the offset from the bid price and ℓ denotes the order polarity (presumed to be $\ell = -1$, denoting a buy order).

Limit sell orders: If the agent has an order above the bid price, and a limit sell order of size k arrives at price $\Delta' \leq \Delta_\theta$, then we consider the agents order matched. In addition, we reduce the size of the order to $k - 1$. A order of unit size would thus not impact the spread.

A limit sell order at price $\Delta' > \Delta_\theta$ does not impact the agents order, and as thus we simply transition the state as usual.

Limit buy orders: If a limit buy order comes in at $\Delta' > \Delta_\theta$, then the agents order is immediately cancelled, i.e $\theta \rightarrow 0$.

Should the buy order decrease the spread, without triggering the automatic cancellation described above, we adjust Δ_θ accordingly. Finally, any limit buy order is placed behind the agents order in the execution queue.

Market sell orders: If the agent has an order at $\Delta_\theta > 0$, then the agents order is given priority. As a result, we reduce the size k of the market order by one unit to $k - 1$.

If the agent has an order at the bid, then priority is given according to the execution queue. Should the agents order be executed as a result of the market order, we again reduce the size by one unit before applying it to the remaining state.

Should the market order deplete the depth at the bid d_B , then Δ_θ is adjusted accordingly.

Cancellations at bid: Should the agent have an order at the bid, $\Delta_\theta = 0$, then we sample which non-agent order is cancelled uniformly. If a cancellation comes in when the depth $d_B = 1$, then we must adjust Δ_θ to reflect the new spread.

Agent market orders: If the agent places a market order when the depth at the ask $d_A = 1$, then the number of empty price levels above the ask is sampled from the estimated parameters. This introduces a small area of *ideal* behaviour around these market orders, which is to the advantage of the optimal strategy. However, in the data observed

$$\mathbf{P}(\delta\Delta = 0) = 1 \quad (\text{B.15})$$

is a good enough approximation that the bias introduced by this is negligible.

B.3 Parameter fitting

After having processed the data as described in Appendix B.1, we have a sequence of events

$$e = [t, s, \theta, s']. \quad (\text{B.16})$$

In general, we will have some chronological sequence $\{e_i\}$ of finite length, which in Section 4.2.2 is always $|\{e_i\}| = 200$. From this sequence we want to estimate the parameters of Section 2.1.

Geometrically distributed variables: The limit order sizes, market order

sizes and the number of empty price levels below/above the bid/ask are modelled to be geometrically distributed, i.e

$$\mathbf{P}(X = k) = (1 - p)^{k-1}p, \quad k > 0. \quad (\text{B.17})$$

The parameter for each of the distributions can be estimated by aggregating all relevant sizes and fitting p using a least-squares method.

Market orders: Market orders arrive with intensities $\lambda_{\mathbf{M}}^{\mathbf{S}}$, $\lambda_{\mathbf{M}}^{\mathbf{B}}$ for sell orders and buy orders respectively. In this case, the intensity is easily estimated by dividing the number of buy/sell market orders n over the time interval δT ,

$$\lambda_{\mathbf{M}} \sim \frac{n}{\delta T}. \quad (\text{B.18})$$

Limit orders & Cancellations: Both limit orders and cancellations of limit orders have intensities λ proportional to some parameter λ_i and some function $f(s)$ of the current state s

$$\lambda = \lambda_i f(s). \quad (\text{B.19})$$

Over the interval δT , the state s will transition at times t_i , such that $t_n - t_0 \leq \delta T$. The net *effect* of the intensity, or rather the expected number of occurrences N , over the time δT is

$$\mathbf{E}[N] = \int_{t_0}^{t_n} dt \lambda_i f(s(t)) = \lambda_i \sum_{i=1}^n (t_i - t_{i-1}) f(s(t_{i-1})). \quad (\text{B.20})$$

We can solve for the parameter λ_i

$$\lambda_i = \frac{\mathbf{E}[N]}{\sum_{i=1}^n (t_i - t_{i-1}) f(s(t_{i-1}))}. \quad (\text{B.21})$$

This process is the same for all limit order and cancellation parameters, though the function $f(s)$ varies as described in Section 2.1.

B.4 Historic data used

For the historic data, quotes and trades for \$AAPL stock from five trading days were selected. Quote indices relative to the beginning of the day were used to

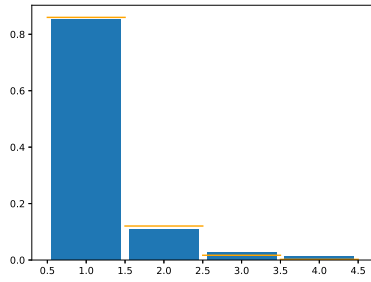
select data, as opposed to times. This ensured that an equal amount of data was selected from all dates, in the first step. The data was obtained through the Polygon API [13].

Episodes were selected from the data such that they were at least $n_{min} = 300$ events long, and fully within the state space defined by the hyper parameters in Section 4.1.3. All such episodes were selected. If an episode was above $2n_{min}$ in size, it was split into multiple episodes of 300 events, with the chronologically last episode containing any remainder. In Figure B.1, we give

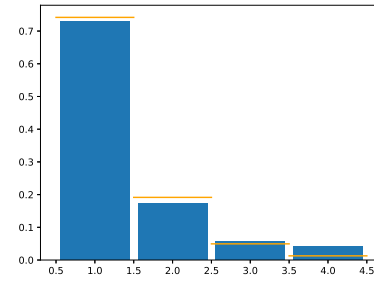
Date	Start index	End index	Number of episodes
03-01-2023	$50 \cdot 10^3$	$450 \cdot 10^3$	153
04-01-2023	$50 \cdot 10^3$	$450 \cdot 10^3$	256
05-01-2023	$50 \cdot 10^3$	$450 \cdot 10^3$	249
09-01-2023	$50 \cdot 10^3$	$450 \cdot 10^3$	90
10-01-2023	$50 \cdot 10^3$	$450 \cdot 10^3$	253

Table B.1: A table over the historic data selected.

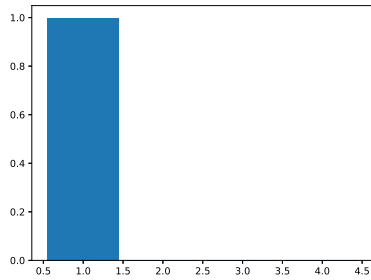
samples of typical distributions for the order sizes, the number of empty price levels, and the depth at the first non empty price level. In Table B.1, details regarding dates and data are given.



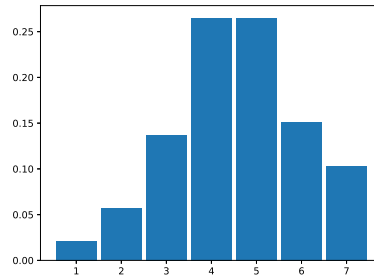
(a) The distribution for sizes of limit orders. A geometric fit is shown in orange.



(b) The distribution for sizes of market orders. A geometric fit is shown in orange.



(c) The distribution for the number of empty price levels, when the bid or ask is emptied, including the recently emptied price level. This was approximated as $\mathbf{P}(\delta\Delta = 0) = 1$.



(d) The distribution for the depth at the next non empty price level.

Figure B.1: A sample of the distributions, from the date *09-01-2023*.

Appendix C

Simple Strategies

In Section 4.2.2, we used two simple strategies as a basis for comparison to the optimal Markov strategy. Here, we will describe them in more detail.

C.1 Market Order

The Market Order strategy amounts to placing a market order at $t = 0$. Since the strategy will always terminate instantly, it is not impacted by the running penalty c . As a result, the strategy's expected episode reward and average execution return are constant as a function of c .

The Market Order strategy can have an arbitrary participation rate r . Consider the following: If we want to purchase n stock with market participation rate r using only market orders then we can start by placing a market order, then waiting $1/r$ transitions before placing the next.

C.2 Bid Plus One

The Bid Plus One strategy is a balance between participation rate and execution price. It takes the following actions:

- If the spread $\Delta > 0$, have a limit order at $\Delta_\theta = 1$.
- If the spread $\Delta = 0$, place a market order.

The choice of the Bid Plus One strategy as a basis for comparison to the optimal Markov strategy is that it approximates the optimal strategy for a running penalty $c \sim 0.05$, whilst still being very simple.

The participation rate and average execution cost are constant as functions of the penalty c . However, the MDP rewards,

$$V(s, \alpha_{\mathbf{BPO}}) = \mathbf{E}[R_{s,s'} + V(s', \alpha_{\mathbf{BPO}})], \quad (\text{C.1})$$

vary accordingly.

